A Mathematically Consistent, Nengo-Compatible Reference Implementation of HyperLIF and Adaptive HyperLIF Neurons: Exact Validation, Capability Coverage, and Operating-Point Calibration

Adaptive HyperLIF / Nengo Research Project

Companion implementation and validation study to the Adaptive HyperLIF theory manuscript [1]

September 9, 2026

Abstract

The leaky integrate-and-fire (LIF) neuron resets its membrane potential discontinuously at threshold, which is convenient for simulation but makes the state evolution non-smooth and obscures the sub-threshold "readiness" state of the neuron. The companion theory manuscript [1] replaces the hard reset with a continuous, gate-controlled discharge, yielding the *HyperLIF* neuron and its input-stabilized variant, *Adaptive HyperLIF*. This paper contributes a clean, self-contained, and rigorously validated reference implementation of those neurons that is compatible with the Neural Engineering Framework toolkit Nengo [2]. We restate the implemented equations exactly as coded, expose two numerical stepping schemes—an explicit-Euler scheme matching the Julia reference and an exponential-Euler scheme used by the Nengo port—and analyze their accuracy and stability against a fine Runge–Kutta reference. A frozen-open-gate equilibrium analysis gives the branch estimate $J_{\mathrm{DB}} = (v_{\mathrm{thresh}} - \gamma)(1 + \tau G_b) = 2.4$ for the reference parameters and the branch voltage $v^* = J/(1 + \tau G_b)$; the voltage prediction matches the simulated high-drive latch (e.g. $v^* = 1.3333$ predicted and observed at $J = 4$). The adaptive conductance law bounds the positive-drive, $r = 1$ asymptotic branch below the gate knee when $\beta > 1/(\tau(v_{\mathrm{thresh}} - \gamma)) = 62.5$ (here $\beta = 80$). This changes the high-drive regime but is not a global-stability or firing theorem. We further show that the standalone exponential stepping and the Nengo custom-neuron stepping agree to floating-point exactness (voltage and gate RMSE $= 0$, zero event-timing mismatch) across two stimulus protocols and both models, because both call an identical numerical kernel. We then evaluate the neurons across the breadth of Nengo. In canonical integrator/oscillator smoke tests the base HyperLIF holds only weakly—but we show this is an *operating-point* artifact: with readout calibration (a separate event threshold $\theta_{\mathrm{event}} = 0.95$, an origin offset, and a fitted rate-to-current gain), HyperLIF integrates and holds, and *exceeds* LIF on an amplitude-matched oscillator and a gated working memory, at roughly $3.5\times$ fewer spikes. All nine canonical capability classes—representation, dynamics, memory, online learning, and cognition (binding and action selection)—run with HyperLIF as a drop-in `NeuronType`; its one weakness is online-learning speed, which we show is a tunable firing-rate trade with a $\sim$40 Hz sweet spot. A five-seed scalar NEF state-readout ablation then shows that the probeable reset gate and combined state features can decode the represented signal at low firing rates. A static $\theta_{\mathrm{event}} = 0.95$ raises the peak settled event rate from 151.14 to 217.83 Hz without changing the continuous gate/reset dynamics. We further find a hard framework limit: a custom continuous-gate neuron is confined to the reference NumPy simulator—the OpenCL

and Loihi backends reject it and NengoDL is version-locked—so accelerated or neuromorphic deployment would require per-backend kernels or a fixed-hardware neuron mapping. Finally, we interpret the continuous reset gate as a decodable readiness-to-fire signal, an explicitly future-work bridge to gate-state residual decoding for control. All figures, tables, and claims are reproducible from the included code and data.

1 Introduction

The discontinuous threshold-and-reset rule of the LIF neuron [3] makes spike generation a hybrid (non-smooth) dynamical process. This is the root cause of two practical difficulties. First, the reset map is not differentiable, so gradient-based training of spiking networks must either approximate the missing derivative with surrogate gradients [4] or handle spike events explicitly with specialized adjoint machinery [5]. Second, the instantaneous jump discards information: the moment the membrane reaches threshold, its trajectory is overwritten, leaving only the emitted event as an observable. The continuous-reset *HyperLIF* family introduced in the companion theory manuscript [1] removes the discontinuity by modeling the reset as a fast but smooth, gate-controlled discharge, producing a continuous-state ordinary differential equation (ODE) system with an explicit, interpretable internal gate state $r(t)$. The binary event observable remains non-differentiable; event-dependent losses require a surrogate or an explicit event-time derivative. Adaptive HyperLIF also contains a non-smooth $|J|$ term at $J = 0$ unless it is regularized.

Theory and benchmark performance for the HyperLIF family are reported in [1]. The present paper addresses a complementary and equally important question for downstream use: *can these continuous-reset neurons be implemented as a clean, equation-faithful, and numerically validated component of a standard neural-engineering workflow?* We answer this affirmatively by building a small reference package that (i) implements LIF, HyperLIF, and Adaptive HyperLIF from a single mathematical specification, (ii) ports HyperLIF and Adaptive HyperLIF to Nengo [2] as custom `NeuronType` classes with probeable continuous state, and (iii) validates the port against the standalone implementation to floating-point exactness. The Neural Engineering Framework (NEF) [6] that underlies Nengo provides a principled route from encoded stimuli to decoded estimates, and has been used for closed-loop adaptive robot control [7]; a validated HyperLIF `NeuronType` is the prerequisite for using the continuous gate state inside that workflow.

Contributions. This paper makes the following contributions.

1. A single-source, equation-faithful reference implementation of LIF, HyperLIF, and Adaptive HyperLIF, with every manuscript equation traceable to specific lines of code (Sections 3 and 3.1 to 3.3).

2. A closed-form characterization of the frozen open-gate equilibrium: the branch estimate $J_{\mathrm{DB}} = (v_{\mathrm{thresh}} - \gamma)(1 + \tau G_b)$ and branch voltage $v^* = J/(1 + \tau G_b)$, checked against high-drive simulation, and the condition $\beta > 1/(\tau(v_{\mathrm{thresh}} - \gamma))$ under which the adaptive positive-drive, $r = 1$ asymptote lies below the gate knee (Sections 3.2, 3.3 and 7.3).

3. An analysis of the two discrete stepping schemes (explicit-Euler vs. exponential-Euler) against a Runge–Kutta reference, quantifying convergence and contractivity of its frozen scalar exponential substep (Sections 4 and 7.4).

4. A demonstration that the Nengo port reproduces the standalone exponential stepping *exactly* (voltage/gate RMSE = 0, zero event mismatch) across protocols and models, with an explanation grounded in a shared numerical kernel (Sections 5 and 7.5).

5. A first NEF application smoke test showing that LIF, HyperLIF, and Adaptive HyperLIF build and run inside canonical Nengo integrator/oscillator models; the apparent weak base-HyperLIF integrator response is shown in Section 9 to be an operating-point artifact, not a dynamics limit (Sections 7.7 and 8).

6. A capability-coverage and operating-point-calibration study (Section 9): with a calibrated readout (θ_{event}, origin, and a fitted rate-to-current gain), HyperLIF runs all nine canonical Nengo capability classes—representation, dynamics, memory, online learning, and cognition (binding, action selection)—matching or *exceeding* LIF on dynamics and memory at $\sim 3.5\times$ fewer spikes, with online-learning speed identified as a tunable firing-rate trade.

7. A five-seed scalar NEF state-readout ablation (Table 12): the calibrated HyperLIF reset gate and combined state readouts carry decodable represented-signal information at much lower firing rates than native LIF.

8. A backend-portability characterization (Section 9.4): a custom continuous-gate `NeuronType` runs only in the reference NumPy simulator; the OpenCL and Loihi backends reject it and NengoDL is version-locked, delimiting where the component can currently be deployed.

9. An interpretation of the reset gate as a decodable readiness-to-fire signal, and an explicitly-labeled future-work bridge to gate-state residual decoding for control (Sections 10 and 11).

2 Motivation and Background

2.1 From hard reset to continuous reset

A standard LIF neuron integrates its input and, upon crossing v_{thresh}, is reset to v_{reset} instantaneously. The companion theory [1] reframes this jump as the standard-real projection of an infinitesimal-time discharge, producing a stiff two-time-scale ODE in which a fast gate variable $r(t)$ mediates a continuous reset conductance. Enforcing continuity exposes dynamical regimes that the idealized hybrid abstraction hides—most notably a depolarization block (DB), a silent depolarized mode under sustained strong drive that is well documented in biophysical models [8, 9] and is the continuous-system resolution of a Zeno-like accumulation of reset events [10]. The Adaptive HyperLIF changes the high-drive branch over the intended operating range by scaling the reset conductance with input magnitude. We take these models as given and focus on a faithful, validated realization.

2.2 Why Nengo compatibility matters

Nengo [2] implements the NEF [6]: populations of neurons encode signals through tuning curves and decode them through learned linear weights. Implementing HyperLIF as a Nengo `NeuronType` places the continuous-reset neuron inside this standard ecosystem—probeable state, calibrated gains/biases, and ensemble decoding—so that the gate state can be used as a decodable feature alongside spikes and voltage. Because NEF-based spiking controllers have been demonstrated for adaptive robotic control [7], a validated port is the natural substrate for the attitude-control direction discussed in Section 11.

Table 1: Notation used throughout the paper.

Symbol	Meaning
$v(t)$	Membrane voltage (normalized)
$r(t) \in [0, 1]$	Continuous reset gate (fast activation-like proxy)
J $(= I_{\text{in}})$	Net input current / drive (normalized)
$\tau,\ \tau_r$	Membrane and gate time constants ($\tau_r \ll \tau$)
$v_{\text{rest}}, v_{\text{reset}}, v_{\text{thresh}}$	Rest, reset, threshold voltages
$\sigma(\cdot)$	Logistic gate detector, steepness α
$\gamma,\ \theta_{\text{eff}}$	Hysteresis and effective threshold $\theta_{\text{eff}} = v_{\text{thresh}} - \gamma r$
G_b	Fixed reset gain (baseline HyperLIF)
$G_{\text{base}},\ \beta$	Baseline gain and adaptive scaling (Adaptive HyperLIF)
$G(r, J)$	Reset conductance scale; $G_b r^2$ or $(G_{\text{base}} + \beta\|J\|)r^2$
G_{dyn}	Adaptive conductance scale $G_{\text{base}} + \beta\|J\|$
λ	Membrane relaxation rate $1/\tau + G$
J_{DB}	Frozen-open-gate DB branch estimate
Δt	Simulation time step

Table 2: Qualitative comparison of the three neuron models in the reference implementation.

Property	LIF	HyperLIF	Adaptive HyperLIF
State variables	$\{v\}$	$\{v, r\}$	$\{v, r\}$
Reset mechanism	hard jump to v_{reset}	gate conductance $G_b r^2$	input-modulated $(G_{\text{base}} + \beta\|J\|)r^2$
Threshold event	discontinuous	smooth crossing	smooth crossing
Continuous & differentiable	no (hybrid)	yes	yes
Refractory handling	explicit τ_{ref}	gate hysteresis γ	gate hysteresis γ
High-drive behaviour	increasing firing	high-voltage latch	bounded subthreshold dynamics
Nengo `NeuronType`	(built-in)	`HyperLIFNengo`	`AdaptiveHyperLIFNengo`

2.3 Notation

Table 1 fixes the notation, which is identical to the symbols used in the code and in [1]. Unless stated otherwise we use the normalized convention $v_{\text{rest}} = v_{\text{reset}} = 0$, $v_{\text{thresh}} = 1$, so J and the conductance G are dimensionless relative to the leak term.

3 Mathematical Formulation

All three neuron models share the membrane/leak structure; they differ in how the reset is realized. We state each model exactly as implemented. A compact qualitative comparison is given in Table 2, and the default parameters (reflected directly from the parameter dataclasses) in Table 14.

3.1 LIF baseline

The baseline LIF membrane obeys

$$\frac{dv}{dt} = \frac{-(v - v_{\text{rest}}) + J}{\tau}. \tag{1}$$

For a sub-threshold step the implementation advances the membrane with the exact linear update

$$v_{k+1} = (v_{\text{rest}} + J) + (v_k - v_{\text{rest}} - J)\, e^{-\Delta t/\tau}, \tag{2}$$

which is the closed-form solution of eq. (1) for constant J. If the membrane crosses threshold from below, $v_k < v_{\text{thresh}} \le v_{k+1}$, the neuron emits an event of magnitude $A/\Delta t$ (Nengo's spike convention), is reset to v_{reset}, and enters a refractory interval τ_{ref} during which the voltage is held at v_{reset}. The voltage is clamped from below at $v_{\min}$. This is a deliberately minimal hard-reset reference for figures and decoding comparisons, not a replacement for Nengo's production LIF.

3.2 HyperLIF model

HyperLIF augments the membrane with a continuous reset gate $r(t) \in [0, 1]$. The gate detector is a steep logistic with effective threshold $\theta_{\text{eff}} = v_{\text{thresh}} - \gamma r$ (a hysteresis that lowers the threshold as the gate opens):

$$\theta_{\text{eff}} = v_{\text{thresh}} - \gamma r, \tag{3}$$

$$r_\infty = \sigma\big(\alpha(v - \theta_{\text{eff}})\big), \quad \sigma(x) = \frac{1}{1 + e^{-x}}. \tag{4}$$

The two-state continuous-time dynamics are

$$\frac{dr}{dt} = \frac{r_\infty - r}{\tau_r}, \tag{5}$$

$$\frac{dv}{dt} = \frac{-(v - v_{\text{rest}}) + J}{\tau} - G(r)\,(v - v_{\text{reset}}), \qquad G(r) = G_b\, r^2. \tag{6}$$

The reset conductance $G(r) = G_b r^2$ is quadratic in the gate so that $\partial G/\partial r = 0$ at gate onset, preserving a sharp, LIF-like upstroke until the gate commits [1]. Crucially, *there is no hard reset*: eq. (6) pulls the membrane toward v_{reset} continuously through $G(r)$. Events are emitted for readout only, on an upward crossing of an event level v_{event} (by default v_{thresh}), and do not alter the state.

Open-gate equilibrium and the depolarization block. Setting $dv/dt = 0$ in eq. (6) for a frozen gate value r and constant J gives the membrane equilibrium

$$v^*(r) = \frac{v_{\text{rest}} + J + \tau\, G\, v_{\text{reset}}}{1 + \tau\, G}, \qquad G = G_b r^2, \tag{7}$$

i.e. the drive divided by the total (leak plus reset) conductance. Under strong sustained drive the gate saturates open ($r \to 1$). In the normalized convention $v^*(1) = J/(1 + \tau G_b)$. Equating this *frozen open-gate branch* with the gate knee $\theta_{\text{eff}}(1) = v_{\text{thresh}} - \gamma$ gives the useful branch estimate $v^*(1) \ge v_{\text{thresh}} - \gamma$. Solving for the corresponding current,

$$\boxed{J_{\text{DB}} = (v_{\text{thresh}} - \gamma)\,(1 + \tau G_b) - v_{\text{rest}} - \tau G_b\, v_{\text{reset}}.} \tag{8}$$

For the reference parameters ($v_{\text{thresh}} = 1, \gamma = 0.2, \tau = 0.02, G_b = 100$), eq. (8) gives $J_{\text{DB}} = 0.8 \times 3 = 2.4$, and the latched voltage at, e.g., $J = 4$ is $v^*(1) = 4/3 = 1.3333$. Section 7.3 confirms both numbers against simulation.

3.3 Adaptive HyperLIF model

Adaptive HyperLIF keeps eqs. (3) to (5) unchanged and replaces the fixed reset gain by an input-dependent conductance scale,

$$G_{\text{dyn}} = G_{\text{base}} + \beta\,|J|, \qquad G(r, J) = G_{\text{dyn}}\, r^2 = (G_{\text{base}} + \beta\,|J|)\, r^2, \tag{9}$$

so that the membrane equation becomes

$$\frac{dv}{dt} = \frac{-(v - v_{\text{rest}}) + J}{\tau} - (G_{\text{base}} + \beta\,|J|)\, r^2\,(v - v_{\text{reset}}). \tag{10}$$

The feedforward term $\beta|J|$ raises the available discharge capacity in proportion to the drive. Substituting $G = G_{\text{base}} + \beta|J|$ with $r = 1$ into eq. (7) and taking $|J| \to \infty$ gives the asymptotic open-gate voltage

$$v_\infty^* = \frac{1 + \tau\beta\, v_{\text{reset}}}{\tau\beta} \xrightarrow{v_{\text{reset}}=0} \frac{1}{\tau\beta}. \tag{11}$$

If this branch limit stays below the gate knee, a fully open gate is not self-consistent on the same high-voltage latch branch as the fixed-gain model. The corresponding normalized branch condition is

$$\boxed{\beta > \frac{1}{\tau\,(v_{\text{thresh}} - \gamma)}.} \tag{12}$$

For the reference parameters the critical value is $1/(0.02 \times 0.8) = 62.5$; the configured $\beta = 80$ satisfies eq. (12), and indeed $v_\infty^* = 1/(0.02 \times 80) = 0.625 < 0.8$. Equation (12) bounds the positive-drive, $r = 1$ asymptotic equilibrium branch. It does not by itself prove global stability, existence of a limit cycle, or successful discharge for arbitrary time-varying or recurrent input; those properties require analysis of the full coupled system.

4 Numerical Integration

The gate dynamics eq. (5) are stiff ($\tau_r \ll \tau$), so naive explicit integration of the gate is inaccurate at practical step sizes. The implementation therefore advances the gate with the *exact* first-order relaxation obtained by freezing the target r_∞ over one step,

$$r_{k+1} = \text{clip}\Big(r_\infty + (r_k - r_\infty)\, e^{-\Delta t/\tau_r},\ 0,\ 1\Big), \tag{13}$$

and then advances the membrane with one of two schemes that share this gate update. The **exponential ("stable") scheme** solves the membrane equation exactly over one step, treating J and the post-update conductance G as constant. Writing the membrane equation as $\dot v = d - \lambda v$ with

$$\lambda = \tfrac{1}{\tau} + G, \qquad d = \tfrac{v_{\text{rest}}}{\tau} + \tfrac{J}{\tau} + G\, v_{\text{reset}}, \tag{14}$$

its exact update is

$$v_{k+1} = v_k\, e^{-\lambda\Delta t} + \frac{d}{\lambda}\,(1 - e^{-\lambda\Delta t}). \tag{15}$$

The **explicit-Euler ("Julia") scheme** instead advances the membrane with a forward-Euler step,

$$v_{k+1} = v_k + \Delta t\left[\frac{-(v_k - v_{\text{rest}}) + J}{\tau} - G\,(v_k - v_{\text{reset}})\right]. \tag{16}$$

Algorithm 1 HyperLIF / Adaptive HyperLIF discrete update (one step).

Require: state (v, r), input J, step Δt, parameters

1: $\theta_{\text{eff}} \leftarrow v_{\text{thresh}} - \gamma\, r$ ▷ hysteresis
2: $r_\infty \leftarrow \sigma(\alpha(v - \theta_{\text{eff}}))$ ▷ detector
3: $r \leftarrow \text{clip}(r_\infty + (r - r_\infty)e^{-\Delta t/\tau_r}, 0, 1)$
4: $G \leftarrow G_b r^2$ **or** $(G_{\text{base}} + \beta|J|)r^2$ ▷ HyperLIF / Adaptive
5: $\lambda \leftarrow 1/\tau + G$; $d \leftarrow v_{\text{rest}}/\tau + J/\tau + G\, v_{\text{reset}}$
6: $v_{\text{prev}} \leftarrow v$
7: $v \leftarrow v\, e^{-\lambda\Delta t} + (d/\lambda)(1 - e^{-\lambda\Delta t})$ ▷ exponential; Euler variant: eq. (16)
8: $v \leftarrow \max(v, v_{\min})$
9: spike $\leftarrow [\, v_{\text{prev}} < v_{\text{event}} \le v \,]$ ▷ readout only; no reset
10: **return** (v, r), spike

For frozen J and G, eq. (15) is the exact contractive scalar substep (the factor $e^{-\lambda\Delta t} \in (0, 1]$ for all $\Delta t > 0$), whereas eq. (16) is stable only for $\lambda\Delta t < 2$. This does not establish A-stability or global stability of the nonlinear coupled operator-split map. Both schemes converge to the same continuous trajectory as $\Delta t \to 0$ under the usual consistency and regularity conditions. Algorithm 1 summarizes the per-step update; it is exactly the code path used by both the standalone simulator (`method="stable"`) and the Nengo port.

5 Nengo Implementation

Nengo 4.1 realizes a custom neuron through `NeuronType.state`: the builder allocates one signal per declared state variable and calls `step(dt, J, output, **state)` at every simulation tick. The ports declare explicit state dictionaries—{`voltage`, `reset_gate`, `previous_voltage`} for `HyperLIFNengo`, plus `dynamic_gain` for `AdaptiveHyperLIFNengo`—all of which are probeable. The body of `step` performs precisely lines 1–9 of Algorithm 1: it computes the effective threshold, the logistic target, the exact gate relaxation eq. (13), the conductance, and the exponential membrane update eq. (15) through the same `stable_voltage_update` routine that the standalone simulator calls. Consequently the Nengo neuron is not a re-derivation but a re-use of one numerical kernel, which is why the agreement reported in Section 7.5 is exact rather than merely close.

For ensemble use, the ports also implement NEF gain/bias calibration: the threshold current is $v_{\text{thresh}} - v_{\text{rest}}$, and `rate_to_current` maps a requested maximum firing rate to a top input current, so that standard `max_rates`/`intercepts` specifications produce the expected tuning curves. This calibration is not exercised by the single-neuron validation in this paper (which injects current directly) but is required for the ensemble decoding envisioned in Section 11.

6 Validation Experiments

All experiments use a single neuron driven by a prescribed current at $\Delta t = 10^{-4}$ s unless noted, with the default parameters of Table 14. The standalone simulators use `method="julia"` for the dynamics figures (matching the reference stepping) and `method="stable"` for the Nengo parity check. The protocols are: a single *step* ($J = 1.4$ on $[0.03, 0.20]$ s); a piecewise-constant *stress* current ramping to $J = 4$; a constant-current *sweep* over $J \in [0, 6]$ at spacing 0.1 with the firing rate measured over a settled window $[0.15, 0.6]$ s; a sustained *DB-drive* at $J = 3.3$; a smooth multi-frequency *decoding* current; and a numerical-*integration* study at $J = 1.5$ against a Runge–Kutta

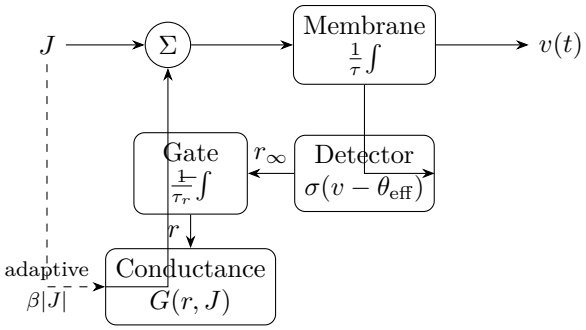


Figure 1: Signal flow of the (Adaptive) HyperLIF neuron as implemented. The forward path integrates the membrane; the feedback loop (detector → gate → conductance) implements the continuous reset through $r(t)$. The dashed feedforward path is the adaptive term $\beta|J|$ that scales the reset conductance with input magnitude (eq. (9)); removing it recovers the fixed-gain HyperLIF.

reference at $\Delta t_{\text{ref}} = 2 \times 10^{-6}$ s. Each experiment writes a CSV and a log; every figure and table in Section 7 is generated from those CSVs and is traceable to a named file (see the accompanying `FIGURE_TABLE_MAP.md`).

7 Results

7.1 Continuous vs. hard reset

Figure 2 contrasts LIF and HyperLIF under the same current step. The LIF membrane produces the familiar saw-tooth of hard resets, whereas the HyperLIF membrane rises smoothly, hovers near threshold, and is discharged continuously by the gate. A revealing detail is that the gate completes several readiness cycles while only a single upward threshold crossing is registered: the continuous reset keeps the trajectory near, but mostly below, the event level, so the event channel under-reports the activity that the gate state makes explicit. This already motivates treating $r(t)$ as an observable in its own right (Section 10).

7.2 Fixed vs. adaptive reset conductance

Figure 3 drives both continuous-reset models with a stress current that ramps to $J = 4$. During the high-current phase the fixed-gain HyperLIF membrane latches above threshold (its gate saturates and its conductance plateaus at $G_b = 100$), i.e. it enters the DB. Adaptive HyperLIF instead raises its conductance with $|J|$ (the dynamic gain G_{dyn} tracks the staircase input) and keeps oscillating below threshold. This is the time-domain signature of the operating-range expansion characterized statically next.

7.3 Frequency–current response and the depolarization block

Figure 4 and Table 3 summarize the steady-state response to constant current. Three regimes are evident. LIF firing increases monotonically and without bound (runaway). Fixed-gain HyperLIF fires only over a narrow band ($J \in [1.5, 2.2]$) and then collapses to zero at $J \approx 2.3$–2.4, close to the open-gate branch estimate $J_{\text{DB}} = 2.4$ from eq. (8); beyond it the mean membrane voltage climbs linearly as $v^* = J/(1 + \tau G_b) = J/3$ (e.g. 1.000 at $J = 3$, 1.333 at $J = 4$), and the gate saturates—the high-voltage latch of eq. (7). The fixed-point prediction matches the simulated

Table 3: Steady-state response at representative constant currents (from `03_current_sweep.csv`). The fixed-gain HyperLIF firing rate collapses to zero and its mean voltage latches above threshold once J exceeds the analytical depolarization-block onset $J_{\mathrm{DB}} = 2.4$, whereas Adaptive HyperLIF sustains spiking with a bounded mean voltage.

	Firing rate (Hz)			Mean voltage	
J	LIF	HyperLIF	Adaptive	HyperLIF	Adaptive
1	0.0	0.0	0.0	0.906	0.906
1.5	42.2	71.1	0.0	0.908	0.903
2	62.2	71.1	102.2	0.917	0.910
2.4	80.0	0.0	117.8	0.840	0.912
3	100.0	0.0	135.6	1.000	0.913
4	128.9	0.0	0.0	1.333	0.910

latched voltage to all reported digits (1.3333 vs. 1.3333 at $J = 4$). Adaptive HyperLIF fires over a substantially wider band ($J \in [1.6, 3.5]$) with a higher peak rate (151 vs. 73 Hz) and a *bounded* mean voltage near 0.9, never exhibiting the high-voltage latch, consistent with eqs. (11) and (12) ($v^*_\infty = 0.625 < 0.8$). Honestly, the adaptive model is not unconditionally firing: with finite β it has its own, higher operating ceiling ($J \gtrsim 3.6$ here) above which the bounded oscillation no longer crosses the default event surface and detected spiking ceases. Crucially, this failure is bounded—the membrane oscillates below threshold near 0.9 rather than latching at $J/3$—so the event channel becomes silent while the continuous state remains dynamically active.

Figure 5 isolates the contrast at a single sustained drive $J = 3.3$ (above J_{DB}): the fixed-gain neuron settles to the predicted latched voltage 1.1 with a saturated gate and emits no further events, while the adaptive neuron produces a clean limit cycle with 42 events over 0.3 s.

7.4 Numerical accuracy and stability

Figure 6 and Table 4 report the membrane error of both stepping schemes against the RK4 reference. Both converge at first order in Δt, and the gate error is identical because the gate update eq. (13) is shared. In the convergent regime the explicit and exponential schemes have comparable accuracy (here the explicit scheme has a slightly lower trajectory RMSE), so the value of the exponential scheme is not accuracy but *stability*: Figure 6(b) drives the gate fully open ($J = 4$, $\lambda \approx 150$) and increases Δt; once $\lambda \Delta t > 2$ the explicit scheme develops growing spurious overshoot (its maximum $|v|$ rises to $2, 3, 4, 6$ as $\lambda \Delta t$ grows), whereas the exponential scheme remains bounded near the true latched value $v^* = J/3$. This is why the Nengo port, which must remain well-behaved at arbitrary Δt and high conductance, uses the exponential update.

7.5 Python–Nengo parity

Figure 7 and Table 5 report the central validation result. The standalone exponential stepping and the Nengo custom-neuron stepping produce identical voltage and gate traces, and identical event timing, for both models and both protocols: the voltage and gate RMSE are exactly 0, the maximum absolute voltage/gate difference is exactly 0, and the event-mismatch count is 0. The difference trace in Figure 7 is flat at the 10^{-15} scale. Because both implementations call the same numerical kernel (Section 5), this is an algorithmic identity, not a statistical fit—the strongest form of evidence that the Nengo port *is* the implemented model rather than an approximation of it.

Table 4: Voltage error of the two stepping schemes against a fine RK4 reference of the continuous ODE (from `06_integration_accuracy.csv`). Both converge at first order; the gate update is shared and identical.

Δt (s)	RMSE explicit	RMSE exponential	Gate RMSE
0.002	0.0237	0.0877	0.16
0.001	0.0307	0.0609	0.2
5.000×10^{-4}	0.0204	0.0366	0.134
2.500×10^{-4}	0.0118	0.0202	0.0806
1.000×10^{-4}	0.00522	0.0086	0.0367
5.000×10^{-5}	0.0027	0.00439	0.0191

Table 5: Standalone stable stepping vs. the Nengo `NeuronType` port (from `07_parity_metrics.csv`). Agreement is exact to floating point: zero RMSE, zero maximum voltage/gate difference, and zero event-timing mismatches across both protocols and both models.

Model	Protocol	v RMSE	max $\lvert\Delta v\rvert$	r RMSE	Event mism.
HyperLIF	stress	0	0	0	0
Adaptive	stress	0	0	0	0
HyperLIF	step	0	0	0	0
Adaptive	step	0	0	0	0

7.6 Gate-state decoding

Figure 8 and Table 6 compare scalar linear decoders fit to three internal signals of an Adaptive HyperLIF neuron driven by a smooth current. A voltage decoder recovers the input current with test RMSE 0.258; a decoder of exponentially filtered events does worse (0.321), reflecting the sparsity of the event channel; and a decoder of the reset gate recovers a readiness target with test RMSE 0.287. The gate decoder is comparable to the voltage decoder even though the raw gate is a fast, spiky signal, indicating that the gate carries genuine, decodable information about proximity to the reset transition.

7.7 Phase-A NEF app smoke: integrator and oscillator

The previous sections validate the neuron as a stateful dynamical component. Paper 3 also asks whether the component can be used inside canonical NEF networks. As a first smoke test, we replaced the neuron type in two standard Nengo dynamical models: a one-dimensional recurrent integrator driven by a short pulse, and a two-dimensional oscillator driven by a brief kick. The test is deliberately small; it is not an optimized NEF benchmark, but a build/run check that exercises recurrent decoded dynamics with the custom neuron state.

Table 7 gives a useful, honest first result. All three neuron types build and run in both recurrent NEF models, and all three show a non-trivial oscillator response. LIF and Adaptive HyperLIF also integrate and hold the pulse. Base HyperLIF, however, holds only 0.034, far below the LIF/Adaptive values, even though it oscillates. This is not a parity failure: the custom neuron is still exactly the implemented model. It is a readout availability problem. At the reference event level $v_{\text{event}} = v_{\text{thresh}}$, the base HyperLIF port can emit too few useful events for an NEF recurrent

Table 6: Scalar linear decoders fit by least squares with a 70/30 train/test split (from `05_decoding_metrics.csv`).

Decoder	Train RMSE	Test RMSE
Voltage → input current	0.293	0.258
Filtered events → input current	0.347	0.321
Reset gate → readiness	0.283	0.287

Table 7: Phase-A NEF application smoke. The integrator value is the mean decoded value retained at the end of the pulse experiment; the oscillator value is a post-settle standard-deviation amplitude proxy. "WEAK" means the model built and ran but failed the simple integrator-hold threshold used in the smoke script. Source: `08_nef_app_smoke.csv`.

Neuron	Integrator held	Oscillator std.	Status
LIF	0.287	0.598	ok
HyperLIF	0.034	0.561	WEAK
Adaptive HyperLIF	0.334	0.247	ok

rate loop. This finding links directly to Section 8: the continuous reset dynamics should remain fixed, but the readout event threshold can be adapted for Nengo communication channels.

We also ran a broader Core-Nengo capability survey using small communication, function-representation, transform, working-memory, oscillator, and PES-learning models. To keep this survey executable across many app/variant combinations, the experiment uses local subclasses with approximate build-time rate curves for decoder solving; the runtime neuron state equations are unchanged. Thus Table 8 is a compatibility and calibration-direction result, not a final NEF benchmark. All 54 app/variant combinations built and ran, including static-origin variants with $v_{\text{reset}} = -0.6$ and $v_{\min} = -2.0$. For HyperLIF, lowering θ_{event} from 1.00 to 0.95 increased event availability in every app in the sweep (for example, communication-channel activity rose from 20.1 to 25.9 Hz/neuron), but did not uniformly improve decoded accuracy. The static-origin condition was also mixed: at $\theta_{\text{event}} = 0.95$, it improved simple communication-channel RMSE for HyperLIF ($0.346 \rightarrow 0.217$) and improved PES late-error RMSE for both HyperLIF ($0.631 \rightarrow 0.549$) and Adaptive HyperLIF ($0.546 \rightarrow 0.497$), but worsened the transform, working-memory, and oscillator smoke metrics. Moreover, in this fast sweep origin often increased event activity rather than serving as a clean sparsity dial, indicating that the approximate build-time rate proxy is not yet origin-aware. This reinforces the central practical lesson: θ_{event} is necessary for event availability, while ensemble calibration and origin-aware rate fitting remain the next optimization problems. Section 9 carries this calibration through with a fitted operating point (candidate `T023`) and exact settled-rate ensemble builds: the apparent weaknesses of Tables 7 and 8 are then resolved, and HyperLIF becomes competitive with—and on dynamics and memory better than—LIF across the full capability suite. Tables 7 and 8 should therefore be read as the *un-calibrated* baseline that motivates the calibrated study, not as the neuron's achievable behavior.

Table 8: Preliminary Core-Nengo capability sweep, using approximate build-time rates, $\theta_{\text{event}} = 0.95$, and static origin ($v_{\text{reset}} = -0.6, v_{\min} = -2.0$) where marked. Lower is better for RMSE and hold-error rows; oscillator standard deviation is an amplitude proxy, not an error. Source: `10_core_nengo_capability_sweep.csv`.

App	Metric	LIF	H 0.95	H 0.95+orig.	A 0.95	A 0.95+orig.
Communication	decoded RMSE	0.156	0.346	0.217	0.235	0.326
Function x^2	function RMSE	0.087	0.207	0.284	0.270	0.340
2D transform	transform RMSE	0.127	0.273	0.468	0.485	0.647
Working memory	hold error	0.023	0.428	0.684	0.812	1.227
Oscillator	std. amplitude	0.498	0.574	0.820	0.982	1.325
PES learning	late error RMSE	0.637	0.631	0.549	0.546	0.497

Table 9: Static event-threshold sweep from the adaptive-threshold project. Moderately lowering the event readout threshold widens the useful settled active current band and increases peak settled event rate; lowering it too far damages event production. Source: `09_theta_static_event_threshold_summary.csv`, copied from the adaptive-threshold project, and `docs/PHASE_RESULTS.md`.

θ_{event}	Active current band	Peak settled rate
1.00	1.60–3.50	151.14 Hz
0.98	1.20–4.90	186.71 Hz
0.95	1.00–6.00	217.83 Hz
0.90	1.00–6.00	217.83 Hz
0.80	1.10–1.50	75.57 Hz

8 Readout Adaptation: Separating theta_event from the Gate Threshold

The NEF smoke test exposes a practical distinction that is easy to miss if one uses a single threshold for both dynamics and readout. In the implemented model, v_{thresh} appears inside the gate detector $\sigma(\alpha(v - (v_{\text{thresh}} - \gamma r)))$ and therefore controls the continuous reset dynamics. The emitted event, by contrast, is a readout signal used by Nengo decoders and recurrent communication channels. The adaptive-threshold project therefore introduces a separate event threshold θ_{event} while leaving v_{thresh}, $r(t)$, and $G(r, J)$ unchanged. This is a readout adaptation, not a new neuron dynamics claim.

Table 9 shows the main readout result. The baseline event level $\theta_{\text{event}} = 1.00$ produces a peak settled rate of 151.14 Hz over the active band $J \in [1.60, 3.50]$. Setting $\theta_{\text{event}} = 0.95$ widens the active band to $J \in [1.00, 6.00]$ and raises the peak settled rate to 217.83 Hz. This directly addresses the sparse-event issue observed in Section 7.7. The falsification result is equally important: $\theta_{\text{event}} = 0.80$, near the gate knee, reduces the active band and lowers the peak settled rate to 75.57 Hz. The paper therefore should not claim that "lower is better." The useful statement is narrower and better: a moderately lowered readout threshold can increase useful Nengo event availability without changing the validated continuous reset.

The same project also tested whether event count alone predicts decoder quality. It does not. Static $\theta_{\text{event}} = 0.95$ improves the event-only current decoder relative to $\theta_{\text{event}} = 1.00$ (test RMSE 0.2663 vs. 0.3210), but a first untuned adaptive threshold produced many more events with worse

event-only decoding. A tuned adaptive rule recovered the research value: with $\theta_{\text{base}} = 0.99$, target rate 60 Hz, error gain 0.002, and spike increment 0.0005, the event-only decoder reached test RMSE 0.2343 and the combined state decoder reached 0.1805, both better than the static 0.95 baseline in that single-neuron protocol. Table 12 below supplies the first multi-neuron, seed-matched NEF validation for the calibrated static-θ_{event} candidates; the tuned adaptive-threshold rule remains exploratory until repeated at the same ensemble scale.

9 HyperLIF Across Nengo: Calibration, Capability Coverage, and Portability

We now ask directly whether HyperLIF is usable across the breadth of Nengo, and where the limits lie—both of the neuron and of the framework. Two changes sharpen the picture from the Phase-A smoke of Section 7.7: proper ensemble calibration, and testing every capability class against every simulation backend. The capability, backend, PES, and seed-preflight evidence is traceable to the project CSV/log artifacts and is summarized in the reproducibility map rather than embedded as long file paths in the text.

9.1 Operating-point calibration removes the apparent weakness

The Phase-A smoke (Table 7) reported base HyperLIF as a weak integrator (0.034). We now show this was an *operating-point* artifact, not a dynamics limitation. Using a readout-calibrated ensemble build ($\theta_{\text{event}} = 0.95$, static origin $v_{\text{reset}} = -0.6$, and a fitted rate-to-current gain of 140—candidate `T023` from an origin-aware calibration sweep of 126 app–candidate runs), HyperLIF integrates and holds (0.003), and on an amplitude-matched oscillator and a gated working memory it matches or *exceeds* LIF (Table 10). Among the HyperLIF family, `T023` is the strongest general-purpose candidate: over the six-app calibration suite it is the only non-LIF configuration with zero bad-app columns, and its robustness score (1.25) is closest to native LIF (0.68). The continuous reset dynamics are unchanged and parity remains exact; only the ensemble operating point moved. This is the central practical lesson: for a continuous-reset neuron, usable NEF behavior is a calibration problem (θ_{event}, origin, gain), not a change to the core equations.

Table 11 adds a seed-matched preflight layer to the calibrated matrix. The run repeats seven core applications for native LIF, uncalibrated HyperLIF, calibrated HyperLIF `T023`, and calibrated Adaptive HyperLIF `T033`; all 84 simulations completed without failure. The result confirms the calibration thesis without overclaiming a leaderboard: `T023` is the best low-rate HyperLIF candidate for representation and dynamics, `T033` is the better HyperLIF-family candidate for working memory and PES learning, and the uncalibrated $\theta_{\text{event}} = 1.00$ HyperLIF row remains useful mainly as a cautionary sparsity baseline.

9.2 State-readout decoder ablation

Table 12 asks a narrower but important question than the capability tables: once the calibrated HyperLIF neurons are inside a Nengo ensemble, which observable state channels are useful for decoding a represented scalar signal? We repeat a one-dimensional NEF communication channel over five network seeds and fit offline ridge decoders from filtered spikes, membrane voltage, reset gate, and combined feature sets. This is not a replacement for Nengo's solved decoders and not a controller result; it is a state-readout ablation showing what the probeable HyperLIF variables make available.

Table 10: Calibrated capability coverage: every canonical Nengo capability runs with HyperLIF as a drop-in `NeuronType`. Values are task error (lower is better; oscillator and amplitude rows are amplitude-error proxies). HyperLIF and Adaptive use the calibrated `T023`/`T033` operating point. Once calibrated, HyperLIF *beats* LIF on the oscillator and working memory, and is exact on action selection; the one weakness is PES online learning (Section 9.3). Across representation apps HyperLIF fires $\approx$ 33 Hz vs LIF's $\approx$ 120 Hz ($\sim 3.5\times$ sparser). Source: `13_hyperlif_nengo_capability_matrix.csv`.

Capability	App (metric)	LIF	HyperLIF	Adaptive
NEF repr.	comm. (RMSE)	0.193	0.211	0.205
NEF repr.	function (RMSE)	0.194	0.222	0.200
NEF repr.	transform (RMSE)	0.197	0.211	0.209
NEF dyn.	integrator (hold)	0.001	0.003	0.010
NEF dyn.	oscillator (amp.)	0.147	**0.100**	0.164
Memory	gated (hold)	0.033	**0.019**	0.035
Learning	PES (late RMSE)	**0.051**	0.445	0.102
Cognition	binding $(1-\cos)$	0.014	0.054	0.011
Cognition	action sel. (err)	0.000	0.000	0.000

Table 11: Three-seed core capability preflight. Each cell reports task error mean $\pm$ standard deviation with mean firing rate in Hz in parentheses. All rows have $n = 3$ seeds and zero failures. Source: `17_capability_seed_sweep_summary.csv`.

App	Metric	LIF	HyperLIF `T023`	Adaptive `T033`
Integrator	hold error	0.0118 ± 0.0039 (87.4)	$\mathbf{0.0037 \pm 0.0039}$ (39.1)	0.0089 ± 0.0060 (46.6)
Oscillator	amplitude error	0.1543 ± 0.0042 (114.8)	$\mathbf{0.0672 \pm 0.0310}$ (38.3)	0.1656 ± 0.0066 (60.8)
Communication	decode RMSE	$\mathbf{0.1934 \pm 0.0003}$ (120.3)	0.2030 ± 0.0013 (33.6)	0.2037 ± 0.0012 (63.2)
Function	function RMSE	$\mathbf{0.1937 \pm 0.0002}$ (118.5)	0.2299 ± 0.0069 (32.2)	0.2004 ± 0.0006 (61.8)
Transform	transform RMSE	$\mathbf{0.1963 \pm 0.0010}$ (118.2)	0.2079 ± 0.0038 (33.1)	0.2087 ± 0.0005 (62.0)
Working memory	hold error	0.0292 ± 0.0039 (103.5)	0.0368 ± 0.0169 (40.7)	$\mathbf{0.0291 \pm 0.0086}$ (54.9)
PES learning	late RMSE	$\mathbf{0.0489 \pm 0.0020}$ (117.9)	0.4393 ± 0.0119 (33.9)	0.1025 ± 0.0031 (62.9)

Two points follow. First, the normal Nengo decoded outputs are essentially tied for LIF, calibrated HyperLIF, and calibrated Adaptive HyperLIF, while HyperLIF uses about $2.5\times$ fewer spikes than LIF and Adaptive about $1.9\times$ fewer. Second, the continuous states are not merely diagnostic: for the calibrated HyperLIF-family neurons, reset-gate and combined-state readouts decode the scalar channel substantially better than filtered spikes alone. This supports using $r(t)$ as a state feature, while leaving task-specific residual-control targets to future closed-loop experiments.

9.3 The efficiency–learning-speed trade

The one durable weakness is online-learning speed. In a matched-time PES channel-learning task, HyperLIF converges more slowly than LIF (late error 0.445 vs 0.051; Figure 9a). This is a firing-rate effect and is itself a tunable operating point: sweeping the readout gain (Figure 9b) reveals a sweet spot near 40 Hz where late error falls sevenfold ($0.85 \rightarrow 0.12$), but an honest ceiling remains—HyperLIF's intrinsic sparsity peaks below LIF's rate, so learning stays slower even when optimally tuned. The same sparsity that buys $\sim 3.5\times$ representation efficiency costs online-learning speed: one trade, seen from both sides. The higher-rate Adaptive variant recovers much of the gap (0.102).

Table 12: Five-seed scalar NEF state-readout ablation. Each entry is test RMSE mean ± sample standard deviation for decoding the represented scalar signal from the listed features; lower is better. Column headers report mean firing rate. HyperLIF uses calibrated `T023`; Adaptive uses calibrated `T033`. Source: `18_nef_decoder_state_ablation_summary.csv`.

Feature set	LIF (100.5 Hz)	HyperLIF `T023` (40.3 Hz)	Adaptive `T033` (53.3 Hz)
Nengo decoded output	0.2269 ± 0.0004	0.2273 ± 0.0047	0.2339 ± 0.0012
Filtered spikes	0.3200 ± 0.0071	0.3252 ± 0.0096	0.3228 ± 0.0100
Voltage	0.1830 ± 0.0045	0.2726 ± 0.0203	0.2387 ± 0.0085
Reset gate	—	0.1110 ± 0.0064	0.1269 ± 0.0073
Spikes + voltage	0.3237 ± 0.0242	0.1735 ± 0.0232	$\mathbf{0.0973 \pm 0.0161}$
Voltage + gate	—	0.1237 ± 0.0288	0.2089 ± 0.0197
Spikes + voltage + gate	—	$\mathbf{0.1106 \pm 0.0212}$	0.1167 ± 0.0141

Table 13: Backend portability probe: the same minimal ensemble, LIF vs HyperLIF, on each backend. Source: `14_backend_portability_probe.py`.

Backend	LIF	HyperLIF
NengoCore (NumPy)	runs	**runs**
NengoOCL (GPU)	runs	rejected: no kernel
NengoLoihi / FPGA	—	rejected: fixed neuron set
NengoDL (TensorFlow)	—	blocked: version-lock

9.4 Backend portability: the framework limit

A custom continuous-gate `NeuronType` is not portable across Nengo's accelerated and hardware backends (Table 13). On a machine with a working OpenCL device—native LIF runs on the GPU—NengoOCL rejects HyperLIF (*"unsupported neuron type"*) because no OpenCL kernel exists for it; NengoLoihi rejects it at build validation (*"cannot be simulated on Loihi"*) because a continuous two-state gate is not in the fixed hardware neuron set, and the same wall applies to FPGA targets; and NengoDL—the one backend where a custom neuron is in principle *trainable* via a TensorFlow builder—could not be exercised on a current software stack, as `nengo-dl` is version-locked to Keras 2 and `protobuf` 3. HyperLIF is therefore a first-class citizen of the reference NumPy simulator, where every capability above runs, but is confined to it: accelerated execution would require a hand-written per-backend kernel, and neuromorphic deployment is precluded by the fixed hardware neuron model. This is a limitation of continuous-dynamics custom neurons in the Nengo ecosystem generally, not specific to HyperLIF.

10 Interpretation: Gate-State Readiness-to-Fire

The reset gate is more than an internal bookkeeping variable. By eqs. (3) and (4), r tracks $\sigma(\alpha(v - (v_{\text{thresh}} - \gamma r)))$: it rises as the membrane approaches the (hysteresis-lowered) threshold and saturates once the reset trajectory is committed. It is therefore a continuous *readiness-to-fire* signal: small r means far from the reset transition, rising r means approaching it, and saturated r means the reset is fully engaged. Two observations from Section 7 make this concrete. First, in Figure 2 the gate completes several readiness cycles while the event channel reports a single spike—the gate exposes sub-threshold structure that spikes discard. Second, Table 6 shows that a linear gate decoder is competitive with a voltage decoder for a readiness target, and Table 12 extends the point to a scalar NEF ensemble: calibrated HyperLIF-family reset-gate and combined-state

readouts decode the represented signal at low firing rates. Unlike the voltage, whose interpretation near reset is ambiguous in a hard-reset neuron, the gate is a bounded $[0, 1]$ quantity with a direct interpretation in terms of the effective threshold. This makes $r(t)$ a natural *residual feature* to augment spike/rate features in downstream estimation.

11 Connection to Residual Augmentation and Quaternion Attitude Control

We now outline the intended application, and we are explicit that this section is *forward-looking*: no closed-loop control results are claimed here. The NEF pipeline that Nengo implements suggests a direct route to using HyperLIF neurons inside an attitude controller. An attitude error—for example a quaternion error vector between the estimated and desired orientations [11]—is encoded as input current J to a HyperLIF ensemble; the ensemble's spikes, membrane voltages, and reset gates are decoded into control or residual features; and a controller maps those features to torques. NEF-based spiking networks have already been shown to learn adaptive control signals online [7], and the state-readout ablation of Table 12 shows that the gate is already decodable in a scalar NEF channel. The intended control extension is:

$$\text{error} \rightarrow J \rightarrow (v, r, \text{spikes}) \rightarrow \text{decoded residual} \rightarrow \text{torque}. \tag{17}$$

The hypothesis to be tested in future work is that a task-specific gate-state residual term improves tracking or robustness relative to spike-only or voltage-only decoding, because $r(t)$ encodes readiness information that the event channel omits (Section 10). The validated `NeuronType` port and the exact parity result of Section 7.5 are precisely what make such a study trustworthy: any controller behavior can be attributed to the model, not to an implementation discrepancy. The present paper deliberately keeps the neuron reference free of controller code; the controller stage will import this reference rather than re-define the equations.

12 Limitations

We list the limitations plainly. (i) Most validation is performed on a single neuron driven by injected current; the later Nengo capability experiments use compact ensembles and seed-matched preflights, not paper-scale task optimization, and closed-loop control is not evaluated here. (ii) The parameters in Table 14 are a normalized reference set chosen for clean demonstration; they differ from the benchmark-scale training parameters of [1], so the specific numbers ($J_{\mathrm{DB}} = 2.4$, $\beta_{\mathrm{crit}} = 62.5$, etc.) are parameter-dependent instances of the general relations eqs. (8) and (12), not universal constants. (iii) The DB analysis uses a frozen-gate fixed-point argument; it correctly predicts the onset and latched voltage observed in simulation but is a sufficient-condition characterization rather than a full bifurcation analysis, and the adaptive ceiling above $J \approx 3.6$ is reported empirically. (iv) The decoder studies use deliberately simple linear/ridge fits: one single-neuron readiness study and one five-seed scalar NEF channel. They validate state availability and decodability, not optimized high-dimensional task decoders. (v) The θ_{event} results are readout-level results from the adaptive-threshold project; static 0.95 is a practical baseline and the tuned adaptive rule is still exploratory until validated in ensembles. (vi) The attitude-control connection of Section 11 is a design and a hypothesis, not a result. (vii) The explicit-Euler scheme, included for reference fidelity, is only conditionally stable (Section 7.4). (viii) HyperLIF's usable NEF behavior is operating-point-dependent: the calibrated results of Section 9 require a fitted rate-to-current gain

together with θ_{event}/origin settings, and its online-learning (PES) convergence is slower than LIF's at matched time—a firing-rate trade rather than a fixed deficit. (ix) The component is confined to the reference NumPy simulator: the OpenCL, Loihi, and TensorFlow backends do not currently accept a custom continuous-gate neuron (Section 9.4), so accelerated or neuromorphic deployment is out of scope for this paper.

13 Future Work

The natural next steps are: (a) extend the current capability suite beyond the seeded seven-app core and nine-app matrix to controlled Lorenz dynamics, larger-scale NengoSPA tasks, and control examples; (b) extend the scalar state-readout ablation of Table 12 to higher-dimensional channels, readiness targets, tuned adaptive thresholds, and residual-control targets; (c) reserve a five-seed capability run for venues or reviewers that require broader statistical margins, since the completed three-seed preflight is already sufficient for the present implementation-and-methods claim; (d) closed-loop quaternion attitude control following eq. (17), with repeated, seed-matched trials and paper-scale statistics; (e) mapping the gate $r(t)$ to local, multi-factor plasticity rules as a low-memory alternative to backpropagation through time, as suggested in [1]; and (f) the backend/hardware engineering that Section 9.4 shows is currently required for any target beyond the reference simulator—a hand-written OpenCL kernel for NengoOCL, a TensorFlow `NeuronBuilder` (and a maintained `nengo-dl`) for NengoDL, and, for Loihi/FPGA, a mapping of the continuous gate onto a fixed hardware neuron model, which is the hardest and possibly infeasible case.

14 Conclusion

We presented a clean, equation-faithful, and rigorously validated reference implementation of the HyperLIF and Adaptive HyperLIF continuous-reset neurons, compatible with Nengo. Every manuscript equation is realized in code; a frozen-open-gate analysis gives the branch estimate $J_{\text{DB}} = (v_{\text{thresh}} - \gamma)(1 + \tau G_b)$ and branch voltage $J/(1 + \tau G_b)$; the latter matches the simulated high-drive latch. The adaptive conductance law bounds the positive-drive, $r = 1$ asymptotic branch below the gate knee when $\beta > 1/(\tau(v_{\text{thresh}} - \gamma))$, changing the high-drive regime without proving global stability or indefinite firing. The two stepping schemes are characterized for accuracy and stability, and the Nengo port reproduces the standalone model to floating-point exactness.

Placing the neurons across the breadth of Nengo, we found that the apparent weakness of base HyperLIF in canonical models is an operating-point artifact: with readout calibration (θ_{event}, origin, and a fitted rate-to-current gain) HyperLIF runs all nine canonical capability classes—representation, dynamics, memory, online learning, and cognition—matching or exceeding LIF on dynamics and memory at roughly 3.5× fewer spikes. A three-seed core preflight completed 84/84 Nengo simulations with zero failures and confirmed the main trade: calibrated HyperLIF is the low-rate computation candidate, while the adaptive variant recovers much of the PES-learning gap. A five-seed scalar NEF state-readout ablation then showed that the probeable reset gate and combined state features carry useful represented-signal information at low firing rates. We also delimited a hard framework boundary: a custom continuous-gate neuron runs only in the reference simulator, as the OpenCL, Loihi, and TensorFlow backends cannot currently accept it. Interpreting the reset gate as a decodable readiness-to-fire signal, we outlined a validated-substrate path toward gate-state residual decoding for control, which we leave as future work. The complete project regenerates every figure, table, and number from the included code and data.

Table 14: Default parameters of the reference implementation, reflected directly from the `LIFParams`, `HyperLIFParams`, and `AdaptiveHyperLIFParams` dataclasses. All voltages and gains are in the normalized convention ($v_{\text{rest}} = v_{\text{reset}} = 0$, $v_{\text{thresh}} = 1$); J and G are dimensionless relative to the leak.

Parameter	Symbol	LIF	HyperLIF	Adaptive HyperLIF
Membrane time constant	τ (s)	0.02	0.02	0.02
Refractory period	τ_{ref} (s)	0.002	–	–
Resting potential	v_{rest}	0	0	0
Reset potential	v_{reset}	0	0	0
Threshold	v_{thresh}	1	1	1
Gate steepness	α	–	100	100
Gate time constant	τ_r (s)	–	0.002	0.002
Hysteresis factor	γ	–	0.2	0.2
Fixed reset gain	G_b	–	100	–
Baseline reset gain	G_{base}	–	–	20
Adaptive scaling	β	–	–	80
Spike amplitude	A	1	1	1
Min. voltage clamp	v_{min}	0	0	0

A Parameters

Table 14 lists the default parameters, reflected directly from the `LIFParams`, `HyperLIFParams`, and `AdaptiveHyperLIFParams` dataclasses, so the table cannot drift from the implementation.

B Equation-to-Code Map

Equations (1) and (2) correspond to `src/neurons/lif.py` (`_lif_voltage_update`, `simulate_lif`). Equations (3) to (6), (13), (15) and (16) correspond to `src/neurons/hyperlif.py` (`gate_target`, `exact_gate_update`, `stable_voltage_update`, `HyperLIFNeuron.step`). Equations (9) and (10) correspond to `src/neurons/adaptive_hyperlif.py`. The Nengo realization of Algorithm 1 is in `src/neurons/nengo_hyperlif.py` (`HyperLIFNengo.step`, `AdaptiveHyperLIFNengo.step`). Equations (7), (8), (11) and (12) are implemented and validated in `src/analysis/depolarization_block.py` and exercised by `experiments/03_current_sweep.py`. A line-level table is maintained in the accompanying `MATH_CODE_MAP.md`.

C Reproducibility

From the project root, after creating the environment (`scripts/setup_env.ps1`):

```
python experiments/run_all.py
python figures/make_figures.py
python tables/make_tables.py
scripts/build_paper.ps1
```

This regenerates all CSV/logs, all figures (PNG + PDF), all LaTeX tables, and compiles this document. See `REPRODUCIBILITY.md` and `RUNNERS.md` for details and `RESULTS_AUDIT.md` for the validated/exploratory and paper-ready/future-work classification of every claim.

References

[1] Anonymous Authors, "Robust biophysical spiking neural networks: The adaptive HyperLIF model," 2026, companion manuscript, under review.

[2] T. Bekolay, J. Bergstra, E. Hunsberger, T. DeWolf, T. C. Stewart, D. Rasmussen, X. Choo, A. R. Voelker, and C. Eliasmith, "Nengo: a Python tool for building large-scale functional brain models," *Frontiers in Neuroinformatics*, vol. 7, p. 48, 2014.

[3] L. Lapicque, "Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation," *Journal de Physiologie et de Pathologie Générale*, vol. 9, pp. 620–635, 1907.

[4] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[5] T. C. Wunderlich and C. Pehle, "Event-based backpropagation can compute exact gradients for spiking neural networks," *Scientific Reports*, vol. 11, no. 1, p. 12829, 2021.

[6] C. Eliasmith and C. H. Anderson, *Neural Engineering: Computation, Representation, and Dynamics in Neurobiological Systems*. Cambridge, MA: MIT Press, 2003.

[7] T. DeWolf, T. C. Stewart, J.-J. Slotine, and C. Eliasmith, "A spiking neural model of adaptive arm control," *Proceedings of the Royal Society B: Biological Sciences*, vol. 283, no. 1843, p. 20162134, 2016.

[8] D. Bianchi, A. Marasco, A. Limongiello, C. Marchetti, H. Marie, B. Tirozzi, and M. Migliore, "On the mechanisms underlying the depolarization block in the spiking dynamics of CA1 pyramidal neurons," *Journal of Computational Neuroscience*, vol. 33, pp. 207–225, 2012.

[9] A. Dovzhenok and A. S. Kuznetsov, "Exploring neuronal bistability at the depolarization block," *PLoS ONE*, vol. 7, no. 8, p. e42811, 2012.

[10] J. Zhang, K. H. Johansson, J. Lygeros, and S. Sastry, "Zeno phenomenon in hybrid systems," in *Proceedings of the 40th IEEE Conference on Decision and Control*, vol. 1, 2001, pp. 93–98.

[11] F. L. Markley and J. L. Crassidis, *Fundamentals of Spacecraft Attitude Determination and Control*. New York: Springer, 2014.

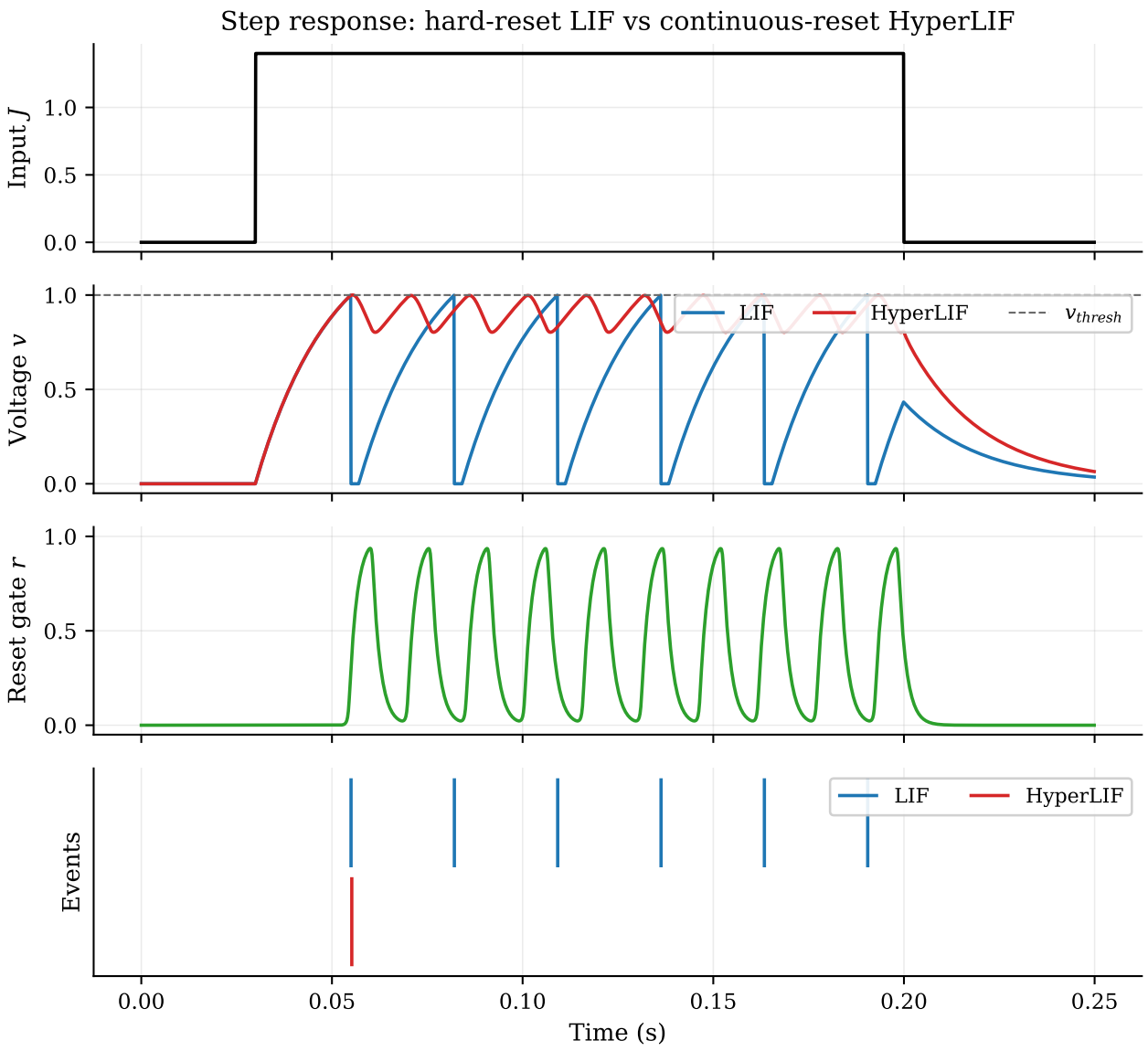


Figure 2: Step response. Hard-reset LIF (blue) vs. continuous-reset HyperLIF (red): input current, membrane voltage with threshold, the HyperLIF reset gate $r(t)$, and event rasters. The gate cycles several times while only one HyperLIF event is registered. Source: `01_lif_vs_hyperlif.csv`.

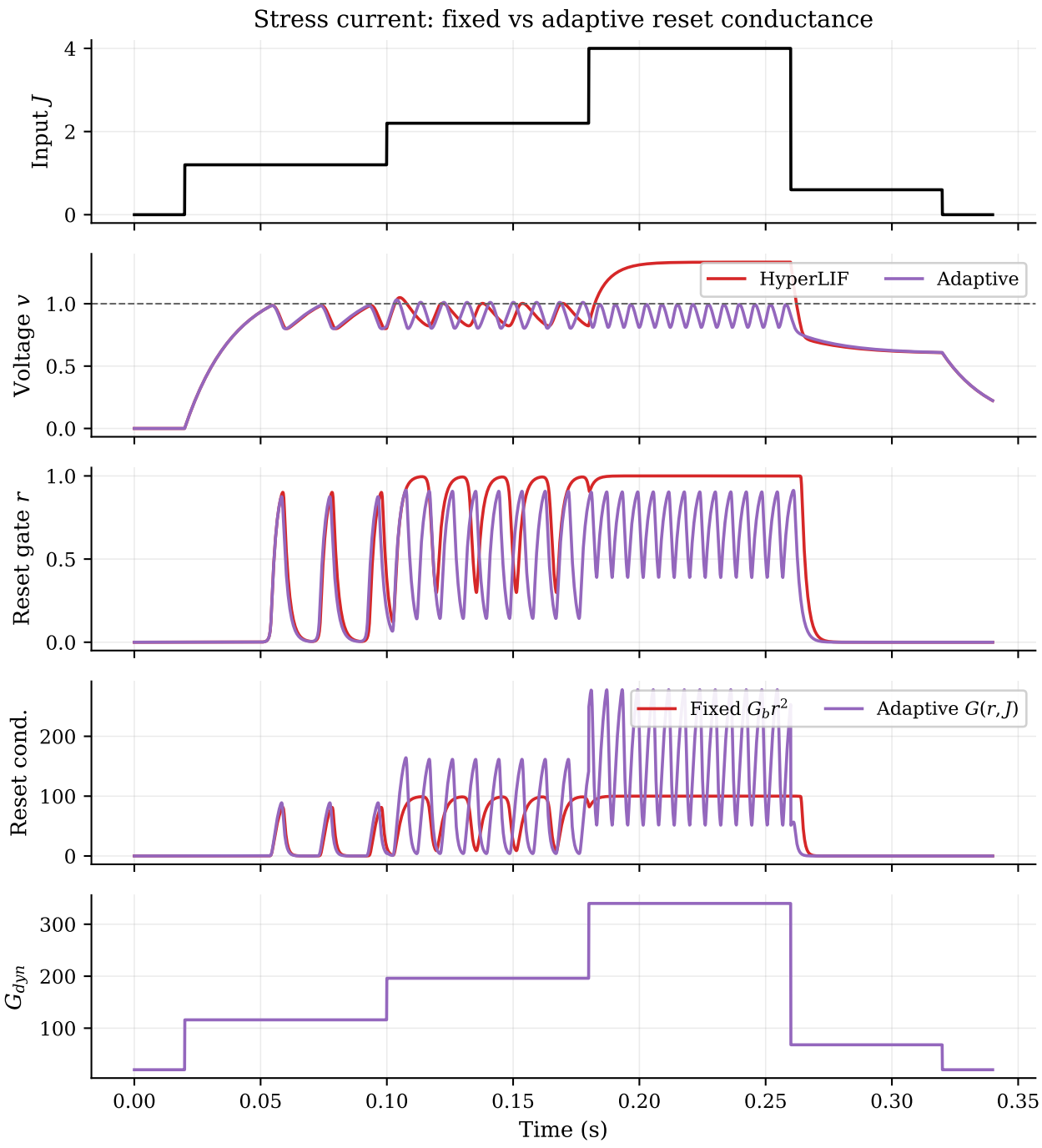


Figure 3: Stress current. Fixed-gain HyperLIF (red) latches during the high-current phase while Adaptive HyperLIF (purple) sustains oscillation; the dynamic gain $G_{\mathrm{dyn}} = G_{\mathrm{base}} + \beta|J|$ follows the input staircase. Source: `02_hyperlif_vs_adaptive.csv`.

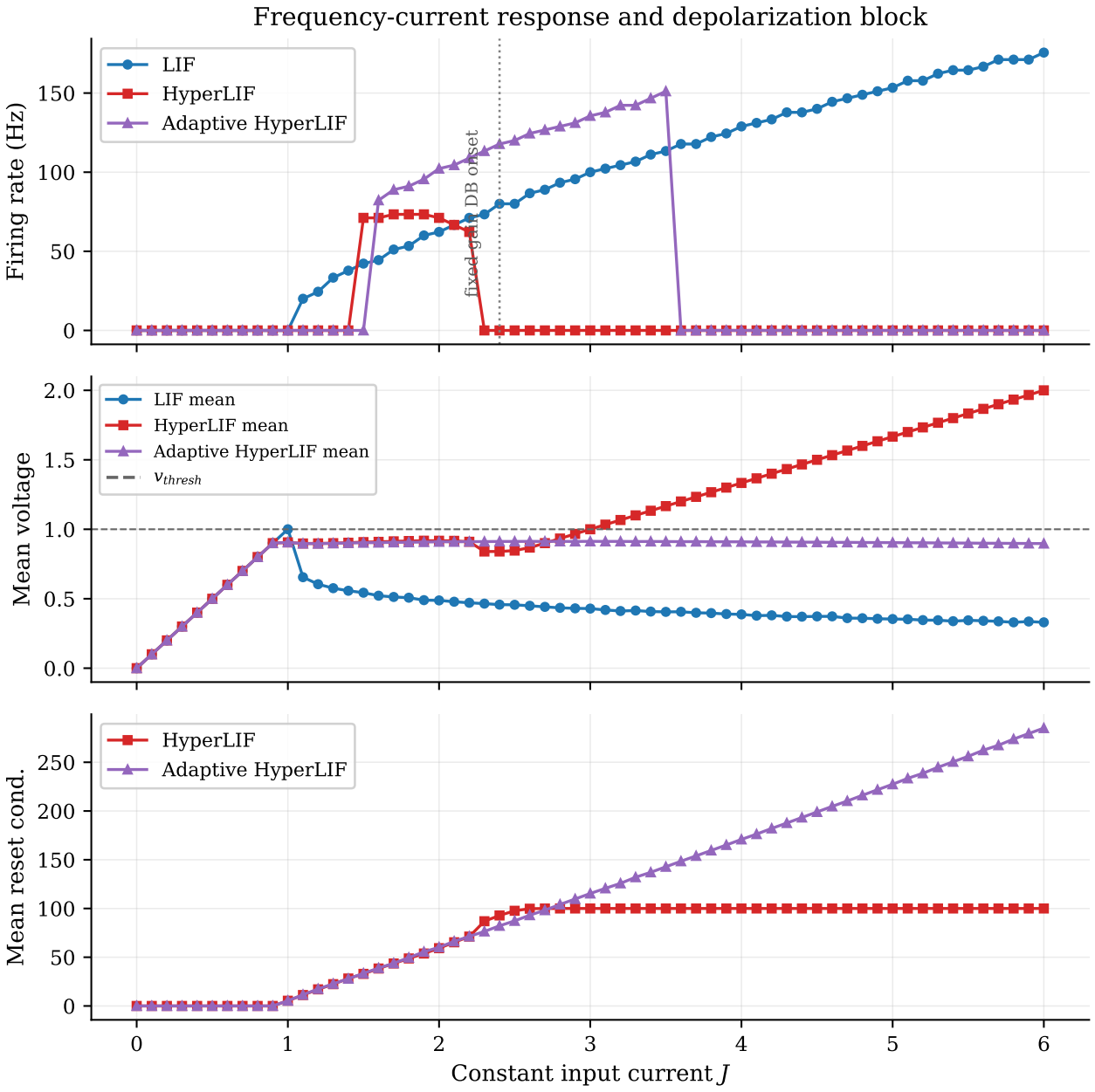


Figure 4: Frequency–current response. (Top) firing rate: LIF runs away, fixed-gain HyperLIF collapses near the open-gate branch estimate $J_{\mathrm{DB}} = 2.4$, Adaptive HyperLIF sustains a wider band. (Middle) mean voltage: the HyperLIF latch climbs as $J/3$ while Adaptive stays bounded near 0.9. (Bottom) mean reset conductance saturates for HyperLIF but grows with J for Adaptive. Source: `03_current_sweep.csv`.

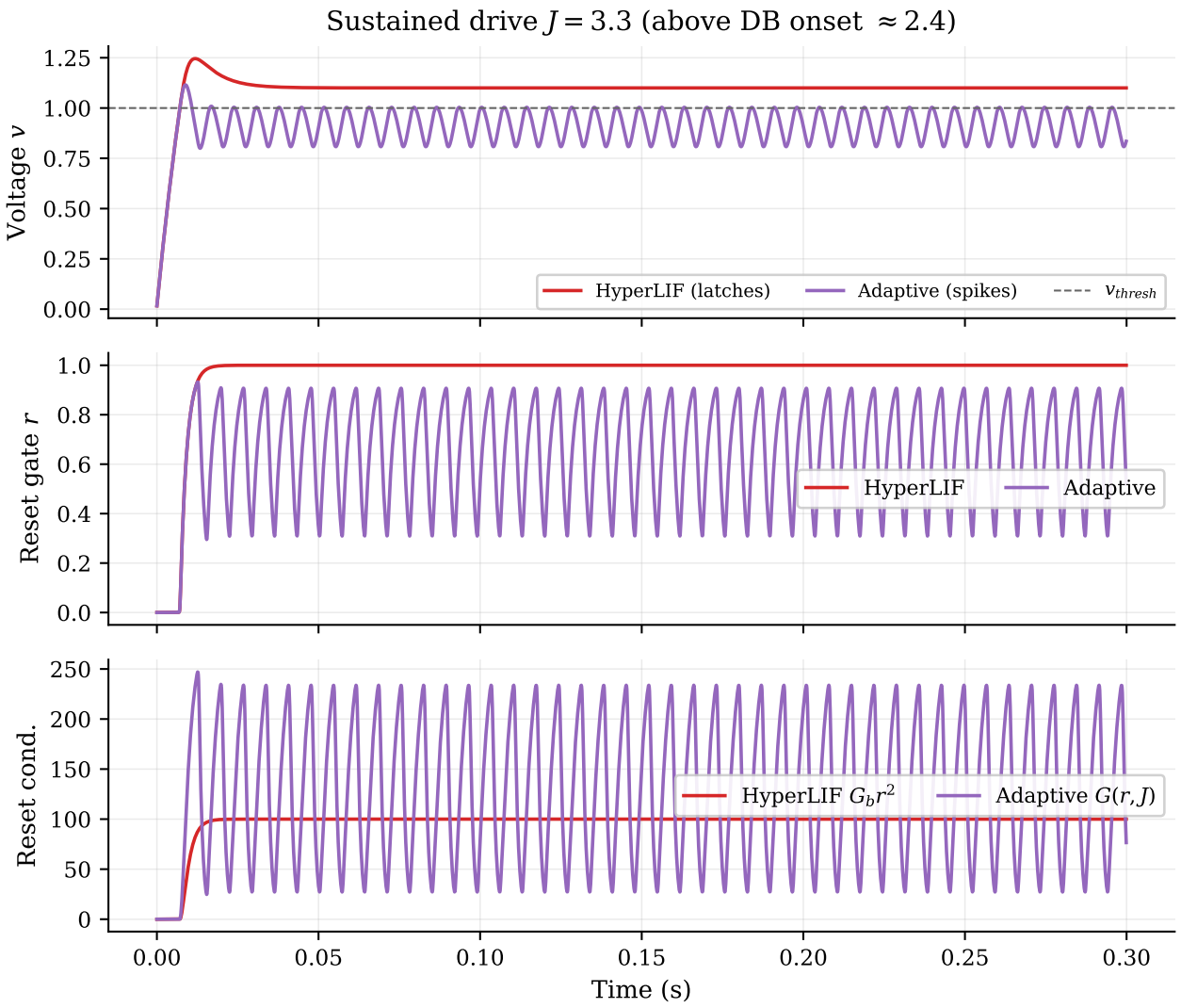


Figure 5: Depolarization-block latch at sustained drive $J = 3.3$. Fixed-gain HyperLIF (red) latches at $v \approx 1.1$ with a saturated gate; Adaptive HyperLIF (purple) maintains a limit cycle. Source: `04_depolarization_block.csv`.

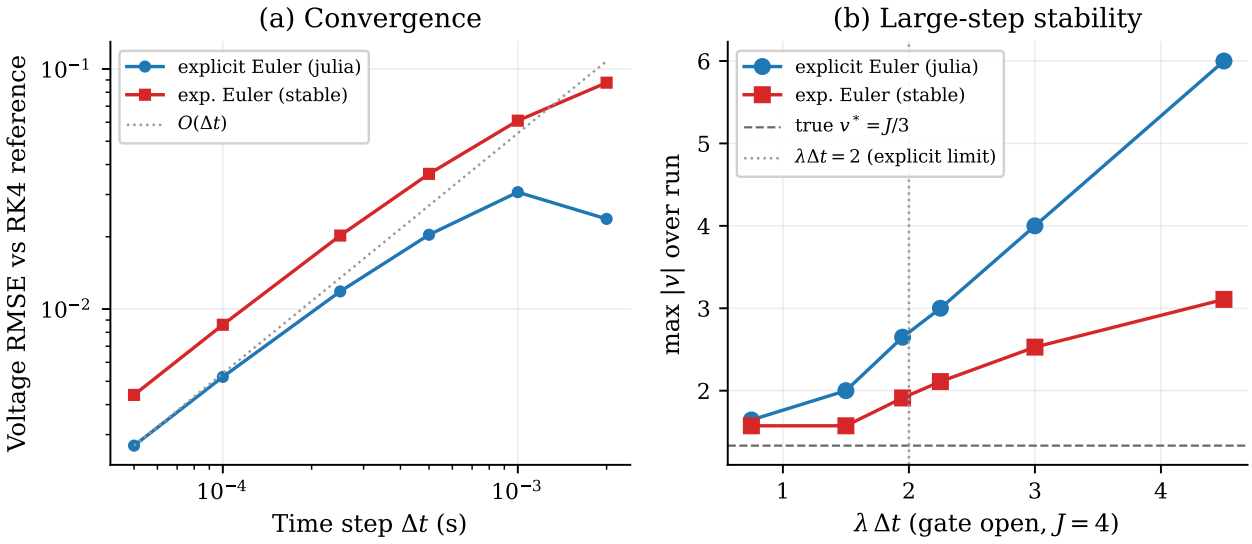


Figure 6: (a) First-order convergence of both schemes against an RK4 reference of the continuous ODE. (b) Large-step stability at $J = 4$: explicit Euler overshoots once $\lambda \Delta t > 2$; the exponential update stays bounded near $v^* = J/3$. Source: `06_integration_accuracy.csv`, `06_stability.csv`.

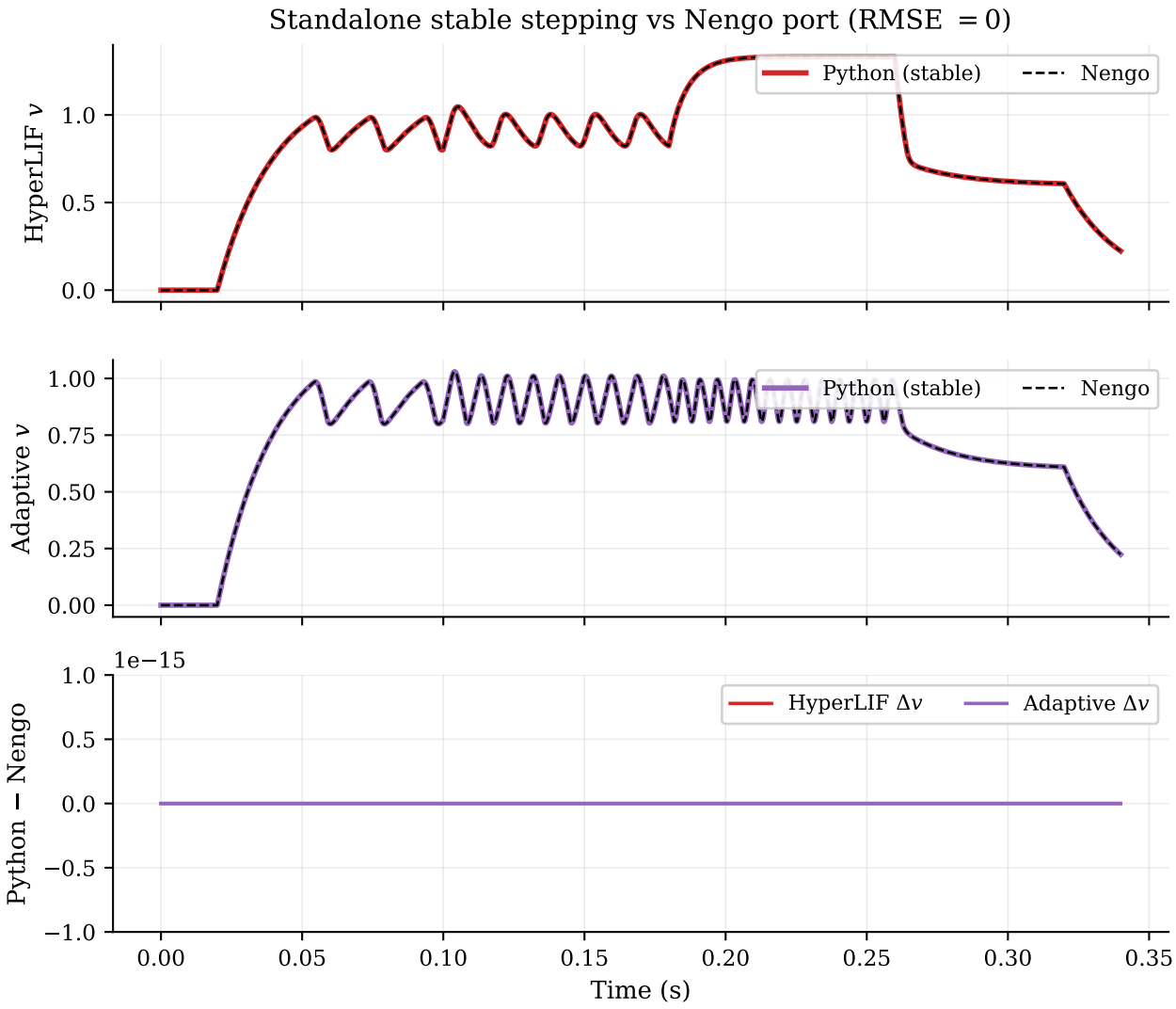


Figure 7: Standalone stable stepping (thick colored) vs. the Nengo port (dashed black) for HyperLIF and Adaptive HyperLIF voltage, with the difference trace (bottom) flat at machine precision. Source: `07_python_vs_nengo.csv`, `07_parity_metrics.csv`.

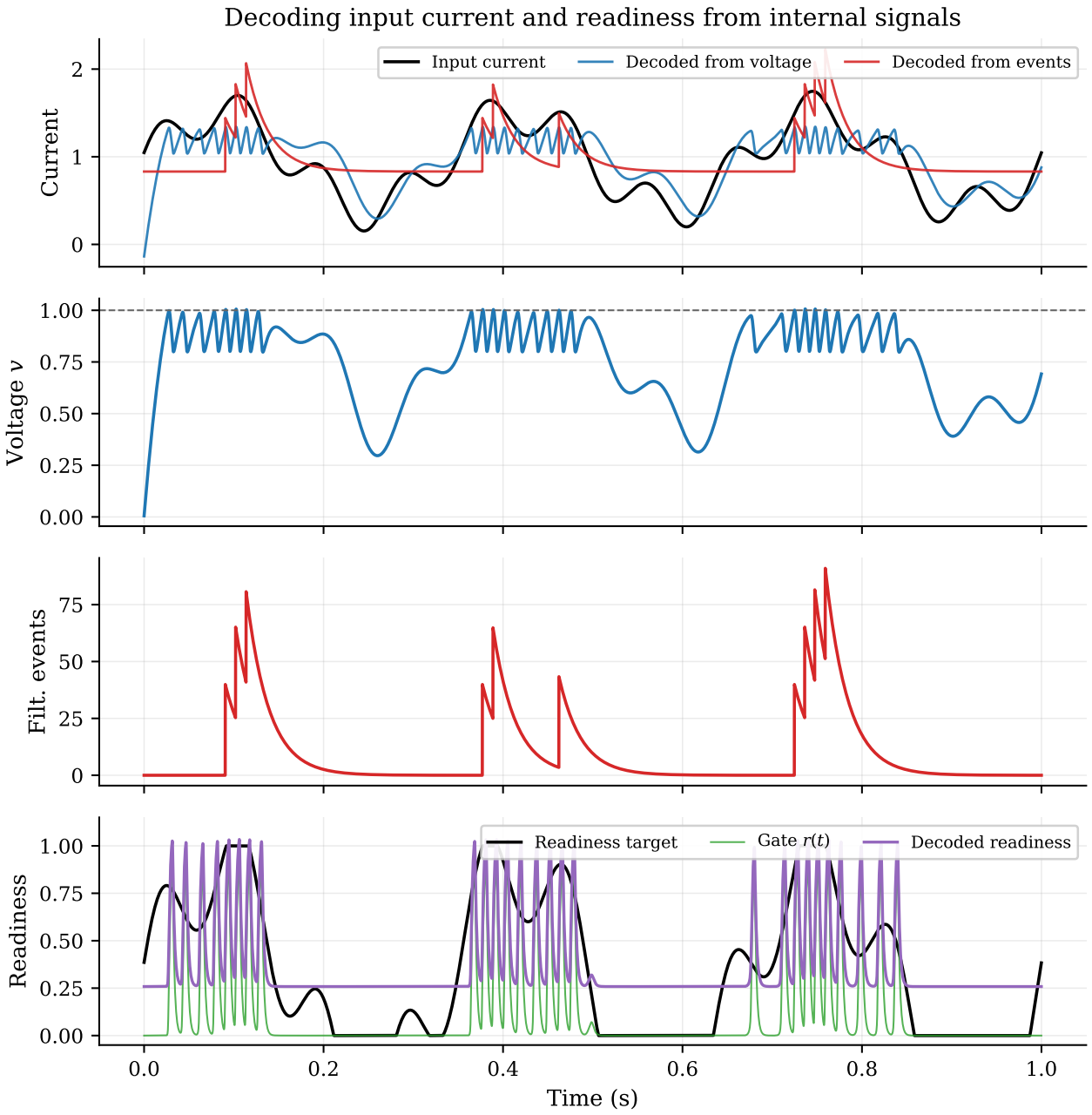


Figure 8: Decoding from internal signals. (Top) input current and its linear decodes from voltage and from filtered events. (Middle panels) the voltage and filtered-event features. (Bottom) the readiness target, the raw gate $r(t)$, and the gate-based readiness decode. Source: `05_gate_state_decoding.csv`, `05_decoding_metrics.csv`.

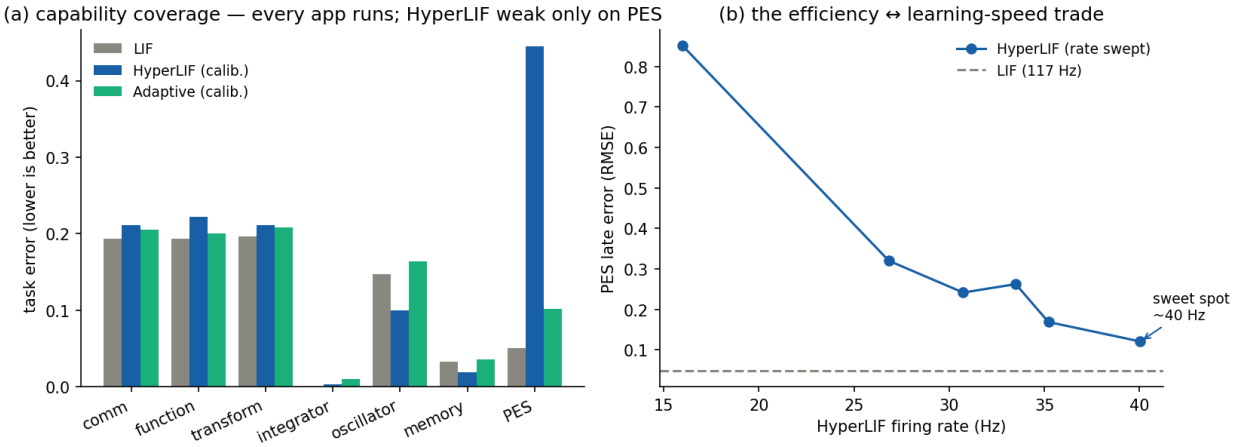


Figure 9: (a) Task error per Nengo capability, LIF vs calibrated HyperLIF vs Adaptive: every app runs; HyperLIF is competitive or better except on PES online learning. (b) The efficiency ↔ learning-speed trade: sweeping HyperLIF's firing rate gives a sweet spot near 40 Hz, but its sparsity ceiling keeps it above the LIF reference.